

# Session 2

## Control Structures & Functions



# Choose one of many, repeat until done, and give your code names.

| Part | Topic                                 | Time  |
|------|---------------------------------------|-------|
| 01   | switch: one choice out of many        | 6 min |
| 02   | while and do-while: repeat until done | 6 min |
| 03   | Writing your own functions            | 7 min |
| 04   | Practice: Digital Dice                | 7 min |
| 05   | AI integration: debouncing the button | 4 min |

## You know these from Lessons 2 to 4.

| Idea                                               | Where                             |
|----------------------------------------------------|-----------------------------------|
| if, else if, else, &&,   , !                       | Lesson 2 · More about programming |
| Reading the button and counting a press on release | Lesson 3 · Programming sensors    |
| for loops                                          | Lesson 4 · Array & for loop       |

Today adds three new tools — switch, while / do-while and your own functions — and puts them to work in a Digital Dice.

# Ten words you will hear today.

| Word         | Plain meaning                                                        |
|--------------|----------------------------------------------------------------------|
| switch       | Picks one block of code to run, by matching a value exactly.         |
| case         | One possible value inside a switch.                                  |
| break        | "Stop here and leave the switch."                                    |
| default      | What runs when no case matches.                                      |
| while        | Repeats while a condition is true — checks first.                    |
| do-while     | Runs once, then repeats while a condition is true — checks after.    |
| Function     | A named block of code you can run from anywhere by calling its name. |
| Parameter    | A value you hand to a function when you call it.                     |
| Return value | The answer a function hands back.                                    |
| Debounce     | Ignoring the tiny flickers a button makes as it is pressed.          |

## Like a vending machine: press 4, get slot 4.

Not "bigger than 3", not "between 2 and 5" — exactly 4. A switch jumps straight to the case that equals the value.

| Use switch when...                     | Use if / else if when...               |
|----------------------------------------|----------------------------------------|
| You match exact values: 1, 2, 3...     | You compare ranges: score $\geq$ 85    |
| There are many choices of one variable | The conditions mix different variables |

# showDice() with a switch.

```
1 void showDice(int value) {  
2     switch (value) {  
3         case 1: Serial.println("one"); break;  
4         case 2: Serial.println("two"); break;  
5         case 3: Serial.println("three"); break;  
6         case 4: Serial.println("four"); break;  
7         case 5: Serial.println("five"); break;  
8         case 6: Serial.println("six"); break;  
9         default: Serial.println("not a dice number");  
10    }  
11 }
```

## LINE 1 · STEP 1 OF 6

showDice takes one number. Say we call showDice(4).

## VARIABLES

value = 4

## SERIAL MONITOR

# showDice() with a switch.

```
1 void showDice(int value) {  
2     switch (value) {  
3         case 1: Serial.println("one"); break;  
4         case 2: Serial.println("two"); break;  
5         case 3: Serial.println("three"); break;  
6         case 4: Serial.println("four"); break;  
7         case 5: Serial.println("five"); break;  
8         case 6: Serial.println("six"); break;  
9         default: Serial.println("not a dice number");  
10    }  
11 }
```

## LINE 2 · STEP 2 OF 6

switch (value) — find the case that equals 4.

## VARIABLES

value = 4

## SERIAL MONITOR

# showDice() with a switch.

```
1 void showDice(int value) {  
2     switch (value) {  
3         case 1: Serial.println("one"); break;  
4         case 2: Serial.println("two"); break;  
5         case 3: Serial.println("three"); break;  
6         case 4: Serial.println("four"); break;  
7         case 5: Serial.println("five"); break;  
8         case 6: Serial.println("six"); break;  
9         default: Serial.println("not a dice number");  
10    }  
11 }
```

## LINE 6 · STEP 3 OF 6

case 4 matches. Print "four"...

## VARIABLES

value = 4

## SERIAL MONITOR

four

# showDice() with a switch.

```
1 void showDice(int value) {  
2     switch (value) {  
3         case 1: Serial.println("one"); break;  
4         case 2: Serial.println("two"); break;  
5         case 3: Serial.println("three"); break;  
6         case 4: Serial.println("four"); break;  
7         case 5: Serial.println("five"); break;  
8         case 6: Serial.println("six"); break;  
9         default: Serial.println("not a dice number");  
10    }  
11 }
```

## LINE 6 · STEP 4 OF 6

...then break jumps out of the switch. Cases 5, 6 and default are skipped.

## VARIABLES

value = 4

## SERIAL MONITOR

four

# showDice() with a switch.

```
1 void showDice(int value) {  
2     switch (value) {  
3         case 1: Serial.println("one"); break;  
4         case 2: Serial.println("two"); break;  
5         case 3: Serial.println("three"); break;  
6         case 4: Serial.println("four"); break;  
7         case 5: Serial.println("five"); break;  
8         case 6: Serial.println("six"); break;  
9         default: Serial.println("not a dice number");  
10    }  
11 }
```

## LINE 10 · STEP 5 OF 6

Leave the switch.

## VARIABLES

value = 4

## SERIAL MONITOR

four

# showDice() with a switch.

```
1 void showDice(int value) {  
2     switch (value) {  
3         case 1: Serial.println("one"); break;  
4         case 2: Serial.println("two"); break;  
5         case 3: Serial.println("three"); break;  
6         case 4: Serial.println("four"); break;  
7         case 5: Serial.println("five"); break;  
8         case 6: Serial.println("six"); break;  
9         default: Serial.println("not a dice number");  
10    }  
11 }
```

## LINE 11 · STEP 6 OF 6

The function ends. A value like 7 or 0 would have matched no case and run default instead.

## VARIABLES

value = 4

## SERIAL MONITOR

four

# Forget a break and the code falls through.

Without break, C++ keeps running into the next case until it meets one. That is almost always a bug — Checkpoint 2 shows what happens.

# One cook checks first. The other tastes first.

"While the water isn't boiling, wait" checks before acting: that is while. "Taste the soup, then decide if it needs salt" acts first: that is do-while.

| Loop     | Checks           | Runs at least once? | Best for                                    |
|----------|------------------|---------------------|---------------------------------------------|
| for      | Before each pass | No                  | Counting a known number of times (Lesson 4) |
| while    | Before each pass | No                  | "Keep going until something happens"        |
| do-while | After each pass  | Yes                 | "Do it once, then repeat if needed"         |

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 1 · STEP 1 OF 12

Start the countdown at 3.

## VARIABLES

count = 3

## SERIAL MONITOR

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 2 · STEP 2 OF 12

Is 3 > 0? Yes — go in.

## VARIABLES

count = 3

## SERIAL MONITOR



# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3   Serial.println(count);
4   count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 3 · STEP 3 OF 12

Print the count.

## VARIABLES

count = 3

## SERIAL MONITOR

3

# A while-loop countdown.

```
1  int count = 3;
2  while (count > 0) {
3      Serial.println(count);
4      count = count - 1;
5  }
6  Serial.println("Roll!");
```

## LINE 4 · STEP 4 OF 12

Take 1 away — otherwise the loop would never end.

## VARIABLES

count = 3 → 2

## SERIAL MONITOR

3

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 2 · STEP 5 OF 12

Back to the check. Is  $2 > 0$ ? Yes.

## VARIABLES

count = 3 → 2

## SERIAL MONITOR

3

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3   Serial.println(count);
4   count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 3 · STEP 6 OF 12

Print it.

## VARIABLES

count = 3 → 2

## SERIAL MONITOR

3  
2

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3   Serial.println(count);
4   count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 4 · STEP 7 OF 12

Take 1 away.

## VARIABLES

count = 3 → 2 → 1

## SERIAL MONITOR

3  
2

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

**LINE 2 · STEP 8 OF 12**

Is 1 > 0? Yes.

## VARIABLES

count = 3 → 2 → 1

## SERIAL MONITOR

```
3
2
```

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3   Serial.println(count);
4   count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 3 · STEP 9 OF 12

Print it.

## VARIABLES

count = 3 → 2 → 1

## SERIAL MONITOR

```
3
2
1
```

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 4 · STEP 10 OF 12

Take 1 away.

## VARIABLES

count = 3 → 2 → 1 → 0

## SERIAL MONITOR

3  
2  
1

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 2 · STEP 11 OF 12

Is  $0 > 0$ ? **No** — skip the loop.

## VARIABLES

count = 3 → 2 → 1 → 0

## SERIAL MONITOR

```
3
2
1
```

# A while-loop countdown.

```
1 int count = 3;
2 while (count > 0) {
3     Serial.println(count);
4     count = count - 1;
5 }
6 Serial.println("Roll!");
```

## LINE 6 · STEP 12 OF 12

Carry on after the loop.

## VARIABLES

count = 3 → 2 → 1 → 0

## SERIAL MONITOR

```
3
2
1
Roll!
```

# A do-while always runs at least once.

```
int n = 10;  
do {  
    Serial.println(n);  
    n = n + 1;  
} while (n < 5);
```

| Line | What it does                                                                 |
|------|------------------------------------------------------------------------------|
| 1    | n starts at 10.                                                              |
| 2–4  | The body runs first: prints 10, then n becomes 11.                           |
| 5    | Now it checks $n < 5$ . 11 is not less than 5, so it stops. It printed once. |

The same loop written with while would print nothing:  $10 < 5$  is false before the first pass.

# A function is a recipe card with a name.

It has a name ("Pancakes"), ingredients you bring (parameters) and a dish you get back (the return value). Once written, anyone can say "make pancakes".

| Part                 | Meaning                                                                |
|----------------------|------------------------------------------------------------------------|
| int (the first word) | The type of the answer it hands back. void means "hands nothing back". |
| addDice              | The function's name — say what it does.                                |
| (int a, int b)       | Two parameters: the ingredients.                                       |
| return a + b;        | Hand back the answer, and leave the function.                          |
| addDice(3, 4)        | The call: 3 goes into a, 4 into b.                                     |

# Calling addDice().

```
1 int addDice(int a, int b) {  
2     return a + b;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int total = addDice(3, 4);  
8     Serial.println(total);  
9 }
```

## LINE 6 · STEP 1 OF 5

The board starts in setup(). Start the Serial monitor.

## VARIABLES

None yet.

## SERIAL MONITOR

# Calling addDice().

```
1 int addDice(int a, int b) {  
2     return a + b;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int total = addDice(3, 4);  
8     Serial.println(total);  
9 }
```

## LINE 7 · STEP 2 OF 5

Call addDice with 3 and 4. Jump up to the function.

## VARIABLES

a = 3

b = 4

## SERIAL MONITOR

# Calling addDice().

```
1 int addDice(int a, int b) {  
2     return a + b;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int total = addDice(3, 4);  
8     Serial.println(total);  
9 }
```

## LINE 2 · STEP 3 OF 5

Work out  $3 + 4$  and hand the answer back with return.

## VARIABLES

$a = 3$

$b = 4$

## SERIAL MONITOR

# Calling addDice().

```
1 int addDice(int a, int b) {  
2     return a + b;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int total = addDice(3, 4);  
8     Serial.println(total);  
9 }
```

## LINE 7 · STEP 4 OF 5

Back in setup(): the answer goes into total.

## VARIABLES

a = 3, b = 4

total = 7

## SERIAL MONITOR

# Calling addDice().

```
1 int addDice(int a, int b) {  
2     return a + b;  
3 }  
4  
5 void setup() {  
6     Serial.begin(9600);  
7     int total = addDice(3, 4);  
8     Serial.println(total);  
9 }
```

## LINE 8 · STEP 5 OF 5

Print it.

### VARIABLES

a = 3, b = 4

total = 7

### SERIAL MONITOR

7

# Press the button, roll a number from 1 to 6.

## ONE LINE OF CODE

```
float pressed = incipe.getButtonResponse();
```

| Button       | You get |
|--------------|---------|
| Pressed      | 1       |
| Not pressed  | 0       |
| Not detected | -1      |

The button needs no data cleaning — it already answers yes or no. Full details: [Wiki › Button](#).

**random(1, 7) gives 1 up to, but not including, 7.**

```
int rollDice(int sides) {  
    return random(1, sides + 1);  
}
```

| Line | What it does                                               |
|------|------------------------------------------------------------|
| 1    | Takes how many sides the dice has; returns a whole number. |
| 2    | random(1, 7) picks 1 to 6. That is why it says sides + 1.  |
| 3    | End of the function.                                       |

Piece 2 — show: showDice(value) from the switch. Piece 3 — the button: count a roll on release, as in Lesson 3. On the LED strip: added once the Wiki lists what its functions take.

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 1 · STEP 1 OF 9

Remember whether the button is being held. At the start it is not.

### VARIABLES

pressed = false

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 4 · STEP 2 OF 9

Read the button. Say it is being pressed.

### VARIABLES

pressed = false

button = 1

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 5 · STEP 3 OF 9

It is 1, so...

### VARIABLES

pressed = false

button = 1

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 6 · STEP 4 OF 9

...remember that it is held. Nothing is rolled yet.

### VARIABLES

pressed = false → true

button = 1

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 4 · STEP 5 OF 9

loop() runs again. Now the button has been let go.

### VARIABLES

pressed = false → true

button = 1 → 0

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 7 · STEP 6 OF 9

It is 0 and it was held — so this is a release.

### VARIABLES

pressed = false → true

button = 1 → 0

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 8 · STEP 7 OF 9

Forget the press, so one press makes one roll.

### VARIABLES

pressed = false → true → false

button = 1 → 0

### SERIAL MONITOR

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 9 · STEP 8 OF 9

Roll a 6-sided dice and show it. Say it rolled 4.

### VARIABLES

pressed = false → true → false

button = 1 → 0

### SERIAL MONITOR

four

## Piece 3 — roll on release.

```
1  bool pressed = false;
2
3  void loop() {
4      float button = incipe.getButtonResponse();
5      if (button == 1) {
6          pressed = true;
7      } else if (button == 0 && pressed) {
8          pressed = false;
9          showDice(rollDice(6));
10     }
11 }
```

### LINE 4 · STEP 9 OF 9

Next pass: still 0, but pressed is false, so nothing happens until the next press.

### VARIABLES

pressed = false → true → false

button = 1 → 0

### SERIAL MONITOR

four

## Put the three pieces in one sketch.

1. Write `rollDice(sides)` and `showDice(value)` at the top.
2. Add `setup()` with `Serial.begin(9600);`
3. Add `bool pressed = false;` and the `loop()` from the walkthrough.
4. Verify, then Upload. Open the Serial monitor at 9600.
5. Press and release the button: one press, one roll. Unplug it: nothing rolls.

# Ask the AI to debounce the button.

Real buttons flicker 1-0-1-0 for a few milliseconds before they settle. That is bounce, and it can turn one press into several rolls.

**Prompt:** My Digital Dice reads `incipe.getButtonResponse()` in `loop()` and rolls when the button is released. Suggest a way to debounce the button. Keep my release logic, explain each line for a beginner, and say what happens if the button returns -1.

# Trust a reading once it has been steady for 50 ms.

```
1  const unsigned long DEBOUNCE_MS = 50;
2  unsigned long changedAt = 0;
3  float lastReading = 0;
4
5  void loop() {
6      float reading = incipe.getButtonResponse();
7      if (reading != lastReading) {
8          changedAt = millis();
9          lastReading = reading;
10     }
11     if (millis() - changedAt > DEBOUNCE_MS) {
12         // steady for 50 ms: trust it here
13     }
14 }
```

| Line  | What it does                                                                                 |
|-------|----------------------------------------------------------------------------------------------|
| 1     | A reading must stay the same for 50 ms.                                                      |
| 2     | When it last changed. millis() counts ms since the board started; it needs an unsigned long. |
| 7–10  | If it changed, note the time and the new reading.                                            |
| 11–13 | Only after 50 ms steady, act on it — put the release logic here.                             |

# What do showDice(3) and showDice(7) print?

### READ THE CODE

```
switch (value) {  
    case 1: ...  
    case 3: Serial.println("three"); break;  
    ...  
    case 6: ...  
    default: Serial.println("not a dice number");  
}
```

### ANSWER

**three**  
**not a dice number**

### WHY

3 matches case 3. 7 matches no case, so default runs.

# case 2 has no break. What does a value of 2 print?

### READ THE CODE

```
switch (value) {  
    case 1: Serial.println("one"); break;  
    case 2: Serial.println("two");  
    case 3: Serial.println("three"); break;  
    default: Serial.println("?");  
}
```

### ANSWER

two  
three

### WHY

Without a break, case 2 falls through into case 3 and stops at its break.

# What does this while loop print?

### READ THE CODE

```
int count = 3;
while (count > 0) {
    Serial.println(count);
    count = count - 1;
}
```

### ANSWER

3

2

1

### WHY

When count reaches 0,  $0 > 0$  is false and the loop stops.

# What does this do-while loop print?

### READ THE CODE

```
int n = 10;  
do {  
    Serial.println(n);  
    n = n + 1;  
} while (n < 5);
```

### ANSWER

10

### WHY

Printed once. A do-while runs its body before it checks; then  $11 < 5$  is false.

## Which numbers can rollDice(6) return?

### READ THE CODE

```
int rollDice(int sides) {  
    return random(1, sides + 1);  
}
```

### ANSWER

**1, 2, 3, 4, 5 or 6**

### WHY

random(1, 7) includes 1 but stops before 7.

# Three things to take away.

1

switch jumps to the case that equals a value. End every case with break, and add a default.

2

while checks first; do-while runs once, then checks; for counts.

3

A function has a name, parameters in and a return value out — so `loop()` can read `showDice(rollDice(6))`.

**Next — Session 3: Arrays, pointers & bitwise operations.** Store ten light readings, find a moving average, and look inside a byte.